

Equality and Transparency

Lean Seminar

Fridolin Weisser

April 14, 2026

This is a short write-up containing all the information from my talk on equality and transparency in the graduate seminar on Lean. If you've got any questions regarding the contents or something is not clear, feel free to contact me at fridolin.weisser@uni-bonn.de.

1 Equality

Equality is one of the most fundamental concepts in Mathematics and therefore also in Lean. If you look for the definition of the equality symbol `=`, you'll see that `a = b` is just notation for `Eq a b`. Herein, `Eq` is an inductive type with the following definition:

```
inductive Eq :  $\alpha \rightarrow \alpha \rightarrow$  Prop where
| refl (a :  $\alpha$ ) : Eq a a
```

We see that the only constructor is `Eq.refl` implementing reflexivity, so for any element `a` of type `$\alpha$` , we have `Eq a a` by `Eq.refl a`. A very useful shortcut is `rfl` given by

```
def rfl { $\alpha$  : Sort u} {a :  $\alpha$ } : Eq a a := Eq.refl a
```

This enables automatic inference of the argument `a`. Let's look at some very basic examples of this definition showcasing the automatic inference and the notation `=`:

```
example : Eq 1 1 := Eq.refl 1
example : Eq 0 0 := rfl
example : 0 = 0 := Eq.refl 0
```

As you might know from your Lean journey, `rfl` doesn't only work for such basic equalities. The following also holds by `rfl` for example:

```
def funny_add (x :  $\mathbb{N} \times \mathbb{N}$ ) := x.1 + 1 + x.2
example (n :  $\mathbb{N}$ ) : let x := 5; funny_add (n, 4) = n + x
:= Eq.refl (n + 5)
```

From the definition of `Eq`, it is not at all clear, why this works. And maybe even more, how we can prove equalities like $(a : \mathbb{N}) + (b : \mathbb{N}) = b + a$ just by using reflexivity. To understand this, we need to look into syntactical equality and definitional equality, two concepts that are a core part of Lean.

1.1 Syntactical Equality

If Lean parses your input, it turns the code into expressions (objects of type `Lean.Expr`), which represent your code as a syntax tree. Two such expressions can now be compared and if Lean finds them as being "equal", we say that they are **syntactically equal**. In particular, syntactical equality doesn't compare two expressions character by character, but rather the syntax trees. In the process of parsing code into a `Lean.Expr`, notation is also expanded. That's why for example `a + b + c` and `(HAdd.hAdd (HAdd.hAdd a b) c)` are also syntactically equal¹, despite not being the same strings.

There is one more aspect to syntactical equality that hasn't been mentioned yet. Two expressions are said to be **α -equivalent** if they are the same up to renaming of binder variables. So for example `fun x => x` and `fun y => y` are syntactically equal.

In conclusion, two expressions are syntactically equal if their syntax trees are the same up to:

- notation (e.g. `Eq 0 0` and `0 = 0`)
- α -equivalence (renaming of binder variables)

To check whether two expressions are syntactically equal, we can rely on the `rw` tactic, since it only works up to syntactical equality.

```
example (x y : ℕ): (fun a => a + x) y = HAdd.hAdd x y := by
  try rw [Eq.refl (Nat.add x y)] -- FAILS
  rw [Eq.refl (fun b => b + x)]
  rw [(Nat.add_comm x y : x + y = y + x)]
```

In this example, the first rewrite tries to find `Nat.add x y` in the goal `(fun a => a + x) y = HAdd.hAdd x y`, which fails. The second rewrite tries to find `fun b => b + x` in the goal, which is syntactically equal to `fun a => a + x`, so this works. And the third looks for `x + y`, which is just notation for `HAdd.hAdd x y`, so this also succeeds.

For more information on syntactical equality, see [1, Part B, Equality] or `src/kernel/expr_eq_fn.cpp` in Lean's source code.

1.2 Definitional Equality

1.2.1 Reduction rules

If syntactic equality were the foundation of Lean's type system, it would be quite limited, as you could only ever prove `a = b` if `a` and `b` are syntactically

¹`+` is infix notation for `HAdd.hAdd`.

equal. Instead, Lean’s type theory is based on something known as **definitional equality**, a less strict notion of equality. Two expressions being definitionally equal is defined via a set of rules that we will go through one by one. The symbol we will use for definitional equality is $\equiv$:

- **Reflexivity:** If e is an expression, then $e \equiv e$.
- **Symmetry:** If $e \equiv e'$, then $e' \equiv e$.
- **Universes:** If we have definitionally equal universe levels $1 \equiv 1'$, then the corresponding universes are definitionally equal: $\text{Sort } 1 \equiv \text{Sort } 1'$.
- **Lambdas:** If we have two definitionally equal types $\alpha \equiv \alpha'$ and two definitionally equal expressions $e \equiv e'$ that may use the variable $x : \alpha$, then $(\text{fun } (x : \alpha) \Rightarrow e) \equiv (\text{fun } (x : \alpha') \Rightarrow e')$. In other words: Lambda definitions respect definitional equality of domain and function term.
- **Extensionality 1:** If we have definitionally equal expressions $f \equiv f'$, $e \equiv e'$, then $f\ e \equiv f'\ e'$.
- **Extensionality 2:** If we have an expression of type $e : (x : \alpha) \rightarrow \beta$ and with a fresh variable $x : \alpha$ we have $e\ x \equiv e'\ x$ for some other expression e' , then $e \equiv e'$.
- **Proof Irrelevance:** If we have two propositions, i.e. $p\ p' : \text{Prop}$, and proofs $h : p$ and $h' : p'$ respectively, and the two propositions are definitionally equal $p \equiv p'$, then the proofs $h \equiv h'$ of these propositions are also definitionally equal. (In short: Proofs of the same proposition are definitionally equivalent.)
- **β -Reduction:** If we apply a lambda function to an argument, this can be reduced: $(\text{fun } (x : \alpha) \Rightarrow e[x])\ y \equiv e[y / x]$, where $e[x]$ is an expression that may depend on the variable x and $e[y / x]$ is the same expression but with every occurrence of x replaced by y .
- **ζ -Reduction:** Reducing let expressions by substitution: $\text{let } x : \alpha := e' \text{ in } e \equiv e[e' / x]$ with the notation $e[e' / x]$ being as above.
- **ι -Reduction 1:** This is difficult to state formally. Informally, it means that if we have a recursor applied to a constructor, this can be reduced by evaluating the appropriate term. More formally, let S be an inductive type with constructors $c_1, c_2, \dots, c_n$. Then

$$(S.\text{rec motive } t_1 \dots t_n)\ (S.c_i\ \text{args} \dots) \equiv t_i\ \text{args} \dots \text{rec} \dots$$

Herein, $\text{args} \dots$ denotes the arguments to the constructor $S.c_i$ and $\text{rec} \dots$ the inductive hypotheses if the constructor has arguments of type S . For a fully formal description of this rule, see [2, Section 2.6].

- **ι -Reduction 2:** This is similar to the first ι -Reduction rule, but for quotients: With the appropriate types `Quot.lift f h (Quot.mk r a) = f a`. See [2, Section 2.7.1] for a formal and [4, `Quot.lift_mk`] for a Lean version.
- **δ -Reduction:** If a definition appears in an expression, δ -Reduction replaces that occurrence by the definition (up to transparency level, which we will cover later).

This description of the reduction rules is not entirely formal and tries to stay in line with Lean syntax. If you want to look into a formal description of all these rules, refer to [2, Chapter 2]. The core implementation of most of these rules can be found in `src/kernel/type_checker.cpp` in the Lean source code.

1.2.2 Examples

After all this theory, let us look into some examples. To see definitional equality in action, we first observe how we can check for it in Lean. Type checking in Lean works up to definitional equality, so if we type check `Eq.refl a : Eq a b` with `a` and `b` being arbitrary expressions, Lean tries to unify the types `Eq a a` and `Eq a b`. By Extensionality 1 and reflexivity, for this it suffices to check `a  $\equiv$  b`. So to check for definitional equality, we can just use `rfl`:

```
example (n : ℕ) : (fun x => x + 0) n = n := rfl

def funny_sum (x : ℕ × ℕ) := x.1 + 1 + x.2
example (n : ℕ) : let x := 5; funny_sum (n, 4) = n + x
:= rfl

def factorial : ℕ → ℕ :=
  Nat.rec 1 (fun n n_factorial => (n + 1) * n_factorial)
example : factorial 5 = 120 := rfl
example : factorial 0 = 1 := rfl
example (n : ℕ) :
  factorial (n + 1) = (n + 1) * factorial n := rfl
```

The first example - albeit looking rather easy - involves already quite a lot of the rules developed above: We first apply β -Reduction to evaluate the function application on the left, which leaves us with `n + 0  $\equiv$  n`. Now, `n + 0` is again a function application (for which we need to δ -reduce the various definitions) and the function is defined recursively on the second argument. Since `0` is notation for the constructor `Nat.zero`, this can be ι -reduced again which finally leaves us with `n  $\equiv$  n`, which is true by reflexivity.

The other examples use similar combinations of reductions, so figuring out which rules are at play is left as an exercise.

1.2.3 Non-Transitivity of Definitional Equality

Already by the first two rules of definitional equality that we presented, it is clear that definitional equality as a relation is reflexive and symmetric. However, we

will now show that it is not an equivalence relation, since transitivity doesn't necessarily hold. One can even show that, if we add transitivity as a rule, definitional equality becomes undecidable, see [2, Section 3.1] for a proof.

For this we need a few lines of Lean code for some definitions:

```
inductive IsTrue (p : Prop) : Prop where
| intro (h : p) : IsTrue p

def outro {p : Prop} : IsTrue p → p
| .intro h => h

def g {p : Prop} : IsTrue p → Nat
| .intro _ => 0
```

We define an inductive type `IsTrue p` with a single constructor `IsTrue.intro`, which takes a proof of a proposition `p` and produces `IsTrue p`. We can now define a destructor-like function `outro`, which takes a `IsTrue p` and produces a proof of `p`. This is possible by using the recursor or - a little easier to read - with pattern matching on `IsTrue p`.

Furthermore, we can define the function `g` on `IsTrue p`, which is zero everywhere, but the definition still uses the recursor of `IsTrue`, even though one could also define it without that. With this set up, we are now ready for the non-transitivity example:

```
example (h : IsTrue True) : g h = g (IsTrue.intro (outro h)) := rfl
example (h : IsTrue True) : g (IsTrue.intro (outro h)) = 0 := rfl
example (h : IsTrue True) : g h = 0 := by
  try rfl -- THIS DOESN'T WORK!!
  match h with | .intro _ => rfl
```

The first definitional equality is by proof irrelevance, as `h` and `IsTrue.intro (outro h)` are of type `IsTrue True`, which lives in `Prop`. The second line is a little more complicated: We have to unfold (δ -reduce) the definition of `g`, see that it is a recursive definition and apply ι -reduction, since we then have a recursor (definition of `g`) applied to a constructor (`IsTrue.intro ...`). This yields 0, so the second equality holds by `rfl`.

The third equality, which would also need to hold by `rfl`, if definitional equality were transitive, cannot just be proven by `rfl`. If we look at the equality, there isn't much in terms of reduction rules that we can do: After unfolding the definition of `g`, we have a recursor applied to some element (not a constructor) and on the other side just a zero. The key here is that we have a "creative" intermediate step in between, which has multiple ways it can be reduced.

1.3 Propositional Equality

As we've seen in the transitivity non-example, but as you probably also noticed while writing Lean code: Not every equality that holds is a definitional equality, i.e. holds by `rfl`. We call this most general version of two things being equal by `Eq a b` **Propositional Equality**. A famous example are the theorems

`Nat.add_zero` and `Nat.zero_add`, the former of which can be proved by `refl` and the latter cannot (instead you have to use e.g. the induction tactic).

1.4 Heterogeneous Equality

1.4.1 Motivation and Definition

If you remember the beginning of this chapter, the notion of equality in Lean is just a (in some sense) arbitrary definition. And one might wonder, if there are limits to this kind of equality or if there are other similar useful notions one can define.

One limitation of the `Eq` definition is that it can only express equality between objects of the same type. So writing `a = b` requires Lean to verify that `a` and `b` are of the same type. This, for example, prevents you from stating equations like `Nat = trivial`. There are also more realistic examples that we will get into below.

To get around this limitation, Lean has a type called `HEq`, short for **Heterogeneous Equality**, which allows to represent equalities between heterogeneous objects (objects having different types). It is defined as follows:

```
inductive HEq : {α : Sort u} → α → {β : Sort u} → β → Prop
where
  | refl (a : α) : HEq a a
```

It also comes with the notation $\asymp$ (typed as `\heq`), so we can now state things like

```
example : (0 : Fin 1)  $\asymp$  (0 : Fin 2) := sorry
```

(even though this is false).

1.4.2 Example

Let us now see this type of equality in action. We define a type `BoundedNat` (`N : ℕ`), which consists of a natural number `n : ℕ` and a proof that `n ≤ N`. While this type probably isn't particularly useful, we can still define an addition on it.

```
structure BoundedNat (N : ℕ) where
  n : ℕ
  hn : n ≤ N

namespace BoundedNat
variable {K L : ℕ}

-- we can define an addition on this family of types
def add (a : BoundedNat K) (b : BoundedNat L)
  : BoundedNat (K + L) where
  n := a.n + b.n
  hn := Nat.add_le_add a.hn b.hn
```

Since the addition is just the one from the natural numbers, restricted to our type, it should inherit the nice properties like having a zero element or being commutative. However, since the addition is not homogeneous, some of these cannot be stated using regular equality.

```
-- Let us define a zero element
def zero : BoundedNat 0 := ⟨0, by trivial⟩

-- We can now state and prove the BoundedNat version of add_zero
theorem add_zero (a : BoundedNat K) : a.add zero = a
  := by simp [add, zero]

-- However, the following fail:
-- zero_add
example (a : BoundedNat K) : zero.add a = a := sorry

-- add_comm
example (a : BoundedNat K) (b : BoundedNat L)
  : a.add b = b.add a := by sorry
```

The error given by Lean for `add_comm` is the following:

```
Type mismatch
  b.add a
has type
  BoundedNat (L + K)
but is expected to have type
  BoundedNat (K + L)
```

However, while these types are not definitionally equal, which Lean complains about, they are certainly propositionally equal. Using `HEq`, we can state and prove these results.

```
example (a : BoundedNat K) (b : BoundedNat L)
  : a.add b ≈ b.add a := sorry

-- To show this, we first show some heterogeneous kind of extensionality
  for BoundedNat
theorem ext {a : BoundedNat K} {b : BoundedNat L}
  (h : K = L) (h' : a.n = b.n) : a ≈ b := by
  cases h; cases a; cases b
  simp at h'
  exact h'

-- this makes proving add_comm quite easy:
theorem add_comm (a : BoundedNat K) (b : BoundedNat L) : a.add b ≈
  b.add a := by
  apply ext
  · apply Nat.add_comm
  · dsimp only [add]
    apply Nat.add_comm
end BoundedNat
```

1.4.3 Limitations

If there are things that one can only state with `HEq`, why keep `Eq` around at all, you might ask. There are mainly two reasons for this. First, while `HEq` can make notation a little simpler, it doesn't actually have more expressive power, as can be seen from the following lemma from `mathlib` (in `Mathlib.Logic.Basic`):

```
lemma heq_iff_exists_eq_cast :
  a ≡ b ↔ ∃ (h : β = α), a = cast h b := sorry
```

So two expressions being heterogeneously equal is really just that their types are equal and the expressions, when casted along this equality, are equal.

And not only is `HEq` not stronger than `Eq`, it additionally cannot guarantee the same congruence rules that `Eq` has:

```
-- HOLDS:
example {f : α → γ} {g : β → γ} {a : α} {b : β}
  (h : f ≡ g) (h' : a ≡ b) : f a = g b := by
  cases h'
  cases h
  rfl

-- DOESN'T HOLD
example {f : α → β} {g : α → γ} {a : α} (h : f ≡ g)
  : f a ≡ g a := sorry
```

The failure of the latter boils down to not being able to prove the following:

```
example (h : (α → β) = (α → γ)) : β = γ := sorry
```

That is why *in general* the use of `HEq` is discouraged. More information can be found in the documentation [4, `Init.Prelude.HEq`].

2 Transparency

We now move on to the second big topic of this talk: *Transparency*. We already mentioned that while learning about δ -reduction, which unfolds definitions if they are "transparent enough". In this section, we will clarify what transparency is, how it interacts with specific tactics and how to change the default behaviour. And finally, we will cover a more recent addition to Lean, namely *modules*, which overlap with transparency in some use cases.

A very rough overview of transparency is the following: Definitions in Lean have a transparency level associated to them. Depending on the context, we unfold up to different levels of transparency. If you think about tensor products, for example, you basically never work with (read: unfold) the concrete definition, most of the time some basic properties like the universal property suffice to work with tensor products. In other cases, such as the definition of a continuous function, we regularly work with (read: unfold) the real definition to prove continuity. With this analogy in mind, it might make sense that we would want to tell Lean about these differences. In Lean, this also has some practical

aspects: During proof automation, unfolding every definition all the time would be very expensive. In other cases (e.g. if Lean has to check if the user's input is correct), we might want to unfold more definitions and try harder to achieve a better user experience. So how do we achieve this in Lean?

2.1 Transparency Levels

In Lean, every definition has a transparency level (`Lean.ReducibilityStatus`) associated to it. There are four different levels:

- **reducible**: This is the "most transparent" level and is the default for `abbrevs` and projections.
- **implicit_reducible**: This is the transparency level for `instances`. We will go into detail as to what this one is for later.
- **semireducible**: This is the default transparency level for definitions. The only exception to this are definitions using well-founded recursion, which fall into the next category.
- **irreducible**: This is the transparency level for definitions using well-founded recursion or `irreducible_def`². It is the "most opaque" level.

To see the reducibility status that a definition has, use:

```
#eval Lean.getReducibilityStatus `Nat.add
```

Note the necessary backtick before `Nat.add`. For `Nat.add`, this would return `implicit_reducible`.

In the above list, we already covered the default transparency level of some ways to write definitions. However, if you are unhappy with the default, you can modify this globally or locally. To globally set the transparency level of a definition, you can prefix the definition with the desired transparency level as an attribute. For example:

```
@[implicit_reducible]
def add_one (n : ℕ) := n + 1
```

If you are fine with the global transparency level of a definition, but want to change it locally (e.g. if you are proving a couple of lemmas for a definition and need to unfold it frequently), you can do so using the `attribute`, `seal` and `unseal` commands. For example:

```
attribute [local reducible] Nat.add

seal add_one -- now irreducible
unseal Real.sqrt -- now semireducible
```

These three commands only affect the current scope.

²This is a mathlib-only command, see here for details.

2.2 Transparency Modes

We now know, how to set transparency levels, so how do they influence tactics and other parts of Lean?

Lean.ReducibilityStatus		.reducible	.implicitReducible	.semireducible	.irreducible
Lean.Meta.TransparencyMode	.none	.reducible	.instances	.default	.all

Figure 1: The relationship between transparency levels and transparency modes.

As you can see in the graphic, Lean defines five transparency modes, ordered linearly. Let's look at these in detail:

- **none**: No definitions are unfolded.
- **reducible**: Only reducible definitions are unfolded. This is the default for speculative tactics like `simp` or `grind`.
- **instances**: This is the default transparency mode when Lean does a type-class search. In addition to **reducible** definitions, this also unfolds those marked as `implicit_reducible`.
- **default**: This is the default transparency mode for most tactics and checking user input. It unfolds everything except for **irreducible** definitions.
- **all**: In this mode, every definition will be unfolded.

Similarly to how we can set non-default transparency levels, we can also instruct Lean to use other transparency modes than it would normally do. For this purpose, there are three tactics: `with_unfolding_none`, `with_reducible` and `with_unfolding_all`. If you prefix a tactic with one of these tactics, the transparency mode will be set to **none**, **reducible** or **all**. One important thing to note is that these don't work on all tactics, e.g. `simp` ignores them, since it relies on internal caches and they would be invalidated by changing this setting.

2.3 Example

We will now see, how transparency levels can be useful when designing a mathematical library. If you've ever looked at the definition of `Set`, you might have noticed the following comment:

```
-- A set is a collection of elements of some type 'α'.
```

```
Although 'Set' is defined as 'α → Prop', this is an
implementation detail which should not be relied on.
Instead, 'setOf' and membership of a set ('∈') should
be used to convert between sets and predicates.
-/
```

```
def Set (α : Type u) := α → Prop
```

Reading this comment, it would've made sense to make `Set` an irreducible definition to enforce a little harder that you shouldn't rely on the definition of set. While such a change would certainly be too disruptive for the ecosystem now, if we were to design the library from scratch, we could try to make use of transparency.

With the current definition, nothing prevents us from relying on the implementation detail:

```
example (a : Set α) : a ⊆ Set.univ := by
  intro x _
  exact trivial
```

However, the intended way to prove this would be:

```
example (a : Set α) : a ⊆ Set.univ := by
  rw [Set.subset_def]
  intro x _
  exact Set.mem_univ x
```

Let us now try to define sets from scratch and make the definition irreducible:

```
@[irreducible]
def Set' (α : Type*) := α → Prop

namespace Set'

variable {α : Type*}

We now want to define some related things like the membership relation or
Set'.univ. For this, we need to use the definition directly, so we unseal it.

---- DEFINITIONS USING PRIVATE Set' DEF ----
unseal Set' in
section

@[irreducible]
def Mem (s : Set' α) (a : α) : Prop := s a

@[irreducible]
def univ : Set' α := fun _ => True

end
```

With this set up, we can define subsets and implement some instances to enable nice notation. Note that we don't have to unseal the definition in this case.

```
---- INSTANCES ----
instance : Membership α (Set' α) where
  mem := Mem

@[irreducible]
def Subset (s1 s2 : Set' α) : Prop
```

```

:= ∀ a : α, a ∈ s1 → a ∈ s2

instance : HasSubset (Set' α) where
  Subset := Subset

To make sure that user code doesn't have to unseal the Set' definitions, we
write some characterizing theorems:

--- Expose defs through theorems ---
unseal Set' Mem univ Subset in
section

@[simp]
theorem mem_univ (a : α) : a ∈ univ := by exact trivial

@[ext]
theorem ext (s1 s2 : Set' α)
  (h : ∀ a : α, a ∈ s1 ↔ a ∈ s2) : s1 = s2 := by
  funext a
  specialize h a
  exact propext h

theorem subset_def (s1 s2 : Set' α) :
  s1 ⊆ s2 ↔ ∀ a : α, a ∈ s1 → a ∈ s2 := by
  constructor <;> exact fun h a ha => h a ha
end

end Set'

```

The above certainly doesn't cover the whole Set-API that the standard library defines, but you can imagine how one would continue. With all this set up, we can now revisit the example from the beginning of this section. Lean now prevents us from unknowingly relying on the definition. Instead we have to use the proper lemmas for that:

```

example (a : Set' α) : a ⊆ Set'.univ := by
  try intro x _ -- FAILS
  rw [Set'.subset_def]
  intro x _
  exact Set'.mem_univ x

```

One final important note: With the current transparency implementation, *reducibility hygiene* is not enforced. This means that when defining other things that rely on this irreducible definition, you should make sure that they are also irreducible (e.g. `Set'.univ` or `Set'.Mem`). Otherwise, the implementation detail might still leak into user code through these.

2.4 Modules

As a final section in this talk, we will briefly introduce modules, a rather new addition to Lean (since version 4.27.0). Since we cannot cover everything in

detail, please read the corresponding language reference entry for everything you need to know.

2.4.1 Introduction

There are several reasons, why the module system was introduced. It helps to separate source files and build units more cleanly, which leads to several build process improvements and speed ups. It also allows for a cleaner separation of a private and a public API of a file, which you might also know from other languages. We will also see how to emulate some of the benefits of transparency with this later.

Currently, the module system is opt-in. You have to write the word `module` at the very top of the file (before imports) to make this source file participate in the module system. Definitions can then either be `public` or `private`, depending on how you prefix them. And you can also decide to expose definitions, if you need to. The module system also heavily influences how you import other files: First, you must only import modules from within modules. For each import, you can then decide whether you want to import only the public (which is the default) or also the private scope of a module. If you only import the public scope of a module, any private definitions can not be unfolded.

2.4.2 Example

To demonstrate that, let's revisit the `Set`-example from before. We now want to redesign `Set` once again, this time using modules. Let's start with the very basic definitions in a file called `AlternativeSet/Defs.lean`:

```
module

import Mathlib.Init

public def Set'' (α : Type*) := α → Prop

namespace Set''
variable {α : Type*}

public protected def univ : Set'' α := fun _ => True

public protected def Mem (s : Set'' α) (a : α) : Prop := s a

public def Subset (s1 s2 : Set'' α) := ∀ a : α, s1 a → s2 a

end Set''
```

Notice the `module` keyword at the top as well as the `public` in front of the definitions. This makes sure that the definitions are available in the public scope of the module, so all importers of this file can use the definitions.

Similar to above, to enable the usual notation for sets, we define some instances in the file `AlternativeSet/Instances.lean`:

```

module

import Mathlib.Init
public import AlternativeSet.Defs

namespace Set''
variable {α : Type*}

public instance : HasSubset (Set'' α) where
  Subset := Set''.Subset

public instance : Membership α (Set'' α) where
  mem := Set''.Mem

end Set''

```

Again, we have the `module`-keyword and make the two instances available to all importers. One new aspect in this file is the `public import`. This is how Lean enforces good API hygiene for the module system. If instead we would do a regular `import` of the `Defs`, it would complain, as the public API of this file (the two instances) contain references to `Set''`, so its definition also has to be part of the public API of this file.

And, to finish this redefinition, we also implement some basic lemmas showing the defining properties of `Set''.univ` and the subset relation in a file called `AlternativeSet/Basic.lean`:

```

module

import all AlternativeSet.Defs
public import AlternativeSet.Instances

namespace Set''
variable {α : Type*} {a b : Set'' α}

@[ext]
public theorem ext (h : ∀ x : α, x ∈ a ↔ x ∈ b) : a = b := by
  funext x
  apply propext
  exact h x

public theorem subset_def : a ⊆ b ↔ ∀ x : α, x ∈ a → x ∈ b
  := by rfl

@[simp]
public theorem mem_univ (x : α) : x ∈ Set''.univ := trivial

end Set''

```

The interesting part in this file is the line with `import all` at the top. As we want to prove properties about the definitions, we need to make use of the

definitions, which requires access to the private part of the `Defs` module. This is possible using `import all`.

Now, to use this new `Set''`, we again prove that any set is subset of the `univ-set`. In a new file, which is also a module (otherwise most features no longer work), we can write the following:

```
module

import AlternativeSet.Basic

example (a : Set''  $\alpha$ ) : a  $\subseteq$  Set''.univ := by
  try intro hx -- we cannot use the private definition
  try unfold Set''.univ -- also fails
  rw [Set''.subset_def]
  intro x _
  exact Set''.mem_univ x
```

The first line, where we want to rely on the private definition of the subset relation, fails as it did in the `Set'` case. The additional guard that the module system provides is that we cannot even unfold private definitions. Instead we use the provided lemmas to prove the statement.

References

- [1] Kevin Buzzard. Formalising mathematics, March 2022. URL: <https://www.ma.imperial.ac.uk/~buzzard/xena/formalising-mathematics-2022/formalisingmathematics.pdf>.
- [2] Mario Carneiro. The type theory of lean. Master’s thesis, Carnegie Mellon University, April 2019. URL: <https://github.com/digama0/lean-type-theory/releases/download/v1.0/main.pdf>.
- [3] The Lean Developers. *The Lean Language Reference*, 4.29.0. URL: <https://lean-lang.org/doc/reference/4.29.0/>.
- [4] The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP ’20. ACM, January 2020.